

Lập trình Java Spring MVC

GV Nguyễn Đức Kiên



Nội dung bài học

- I. Giới thiệu về Spring và Spring MVC
- II. Bean trong Spring
- III. Dispatcher-servlet trong spring
- IV. Resolving Views trong Spring MVC
- V. Controller trong Spring MVC

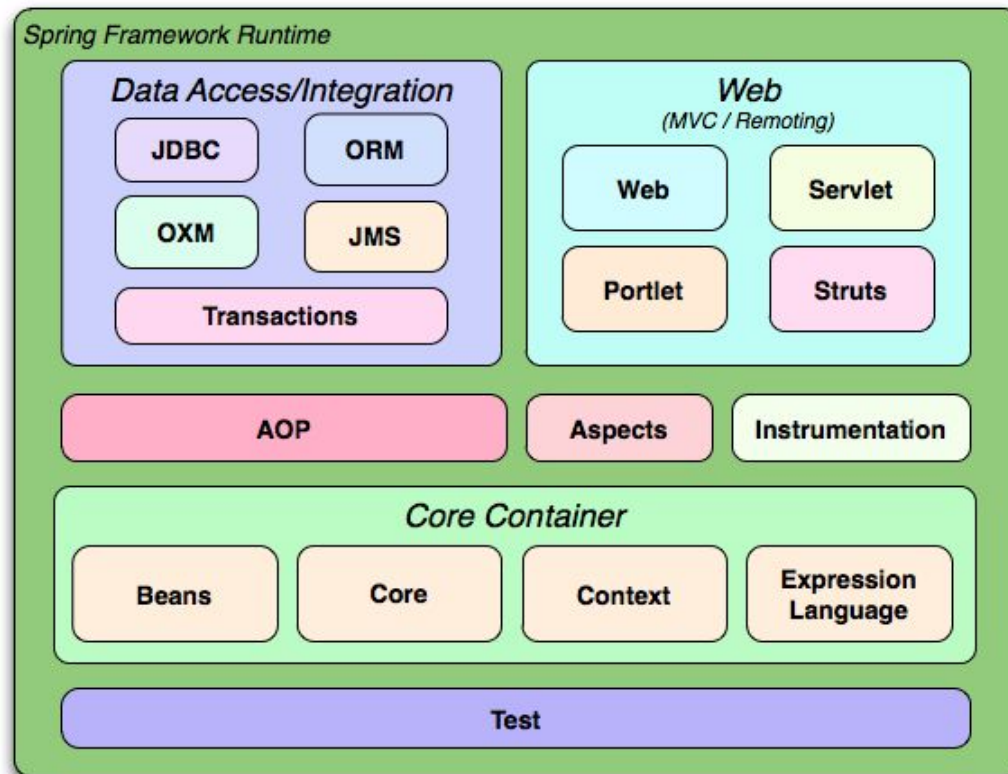
1) Giới thiệu về Spring và Spring MVC

1.1 Giới thiệu về Spring và hệ sinh thái Spring

1.2 Luồng xử lý MVC trong Spring MVC

1.1 Giới thiệu về Spring và hệ sinh thái Spring

Spring là một framework mã nguồn mở rất phổ biến, được thiết kế để giúp phát triển ứng dụng Java dễ dàng và linh hoạt. Các tính năng chính của Spring bao gồm:



1.1 Giới thiệu về Spring và hệ sinh thái Spring

- **Dependency Injection (DI):** Giúp tiêm phụ thuộc vào các lớp mà không cần phải tạo đối tượng trực tiếp trong lớp đó, giúp giảm sự phụ thuộc và tăng tính tái sử dụng của mã nguồn.
- **Inversion of Control (IoC):** Spring quản lý vòng đời của các đối tượng (beans), giúp bạn chỉ cần khai báo và Spring lo việc khởi tạo và quản lý chúng.
- **Aspect-Oriented Programming (AOP):** Giúp tách các phần chức năng cross-cutting (ví dụ như logging, transaction management) khỏi các lớp nghiệp vụ chính.

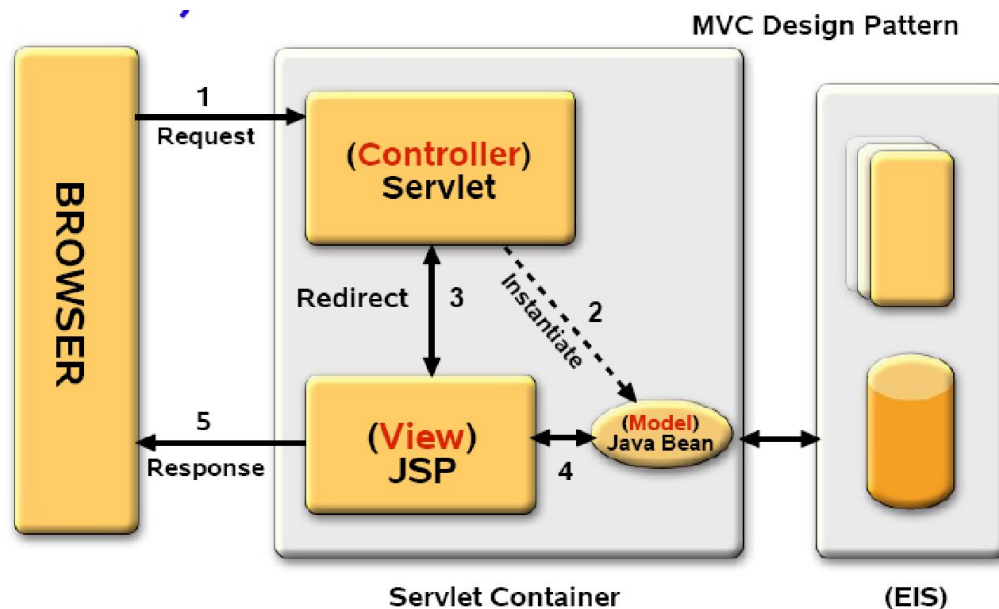
Lý do cần Spring thay vì chỉ sử dụng Servlet

- Spring cung cấp một môi trường quản lý các đối tượng (bean) và tiêm phụ thuộc, làm giảm mã lặp lại và dễ bảo trì.
- Spring MVC hỗ trợ mô hình MVC, giúp tách biệt rõ ràng giữa các tầng của ứng dụng (Model, View, Controller).

1.2 Luồng xử lý MVC trong Spring MVC

Spring MVC sử dụng mô hình Model-View-Controller (MVC), giúp phân tách rõ ràng các thành phần trong ứng dụng:

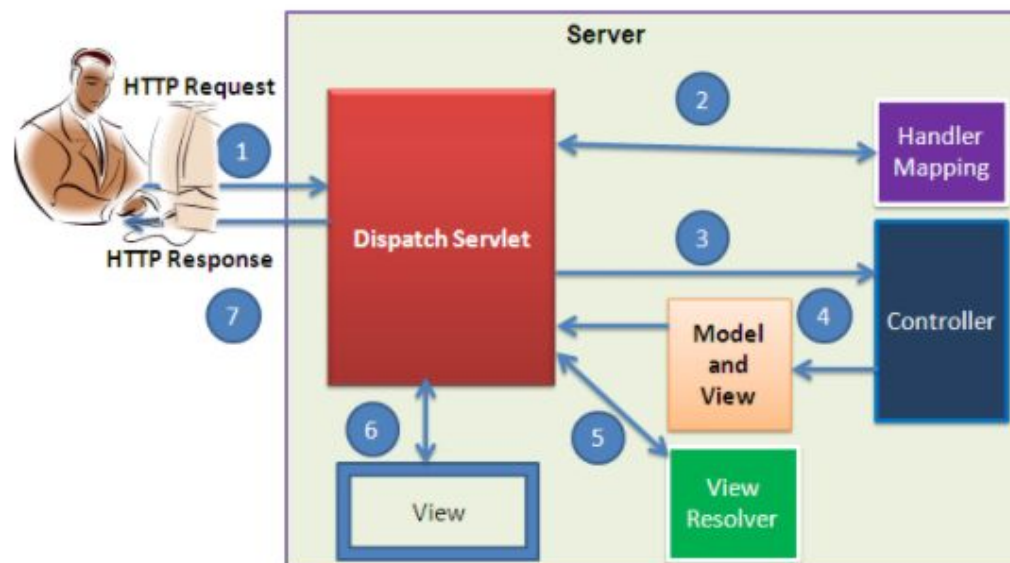
- **Model:** Đại diện cho dữ liệu của ứng dụng.
- **View:** Là phần giao diện người dùng (thường là JSP, Thymeleaf, v.v.).
- **Controller:** Xử lý các yêu cầu HTTP từ người dùng, thực hiện các logic nghiệp vụ và trả về kết quả.



1.2 Luồng xử lý MVC trong Spring MVC

Luồng xử lý trong Spring MVC:

1. DispatcherServlet nhận yêu cầu HTTP từ người dùng.
2. Controller xử lý yêu cầu và trả về model dữ liệu.
3. ViewResolver xác định và trả về view tương ứng.
4. View (thường là JSP) hiển thị kết quả cho người dùng.



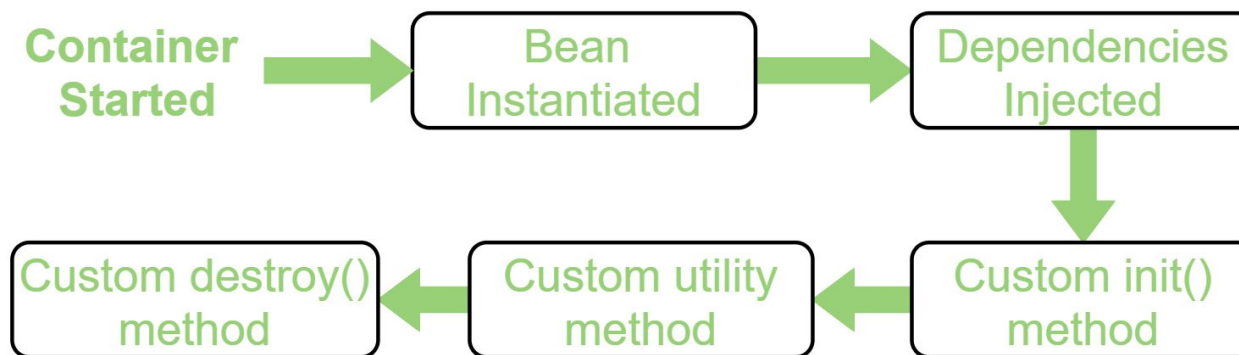
II) Bean trong Spring

2.1 Định nghĩa Bean và cách config

2.2 context:component-scan

2.1 Định nghĩa Bean và cách config

- Bean trong Spring là các đối tượng mà Spring Container quản lý. Bạn có thể khai báo các bean trong dispatcher-servlet.xml để Spring tự động khởi tạo và quản lý.



2.1 Định nghĩa Bean và cách config

Cách config bean

id="studentService": Tên của bean.

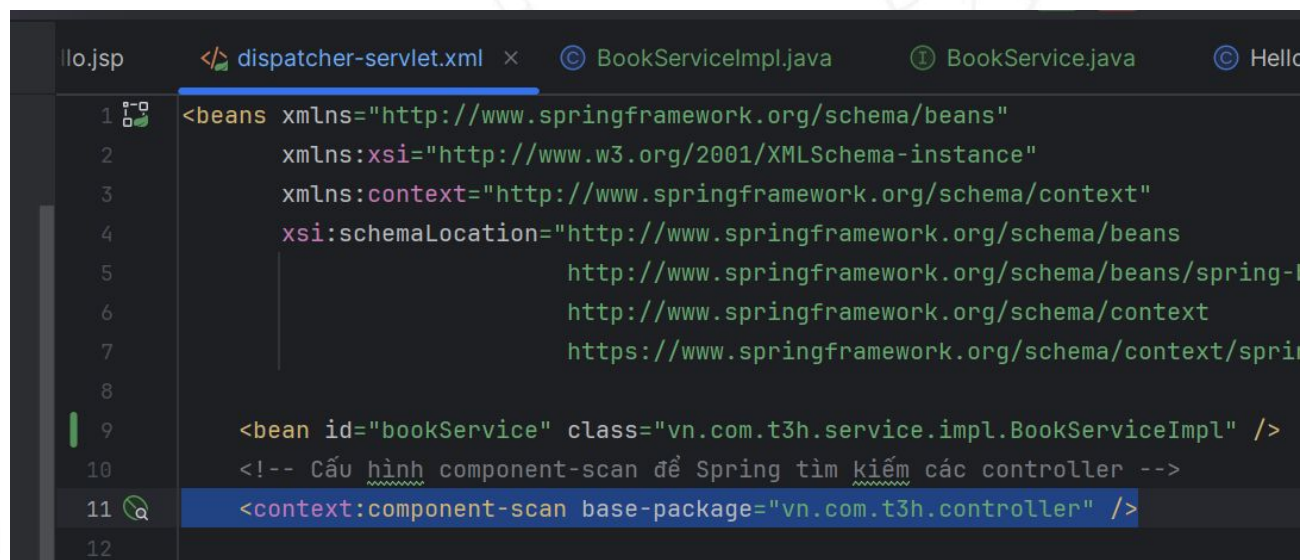
class="vn.com.t3h.service.impl.BookServiceImpl": Lớp Java mà Spring sẽ tạo ra và quản lý.

```
4 import vn.com.t3h.service.BookService,  
5 @Service new *  
6 public class BookServiceImpl implements BookService {  
7 }  
8 |
```

```
o.jsp  </> dispatcher-servlet.xml x  BookServiceImpl.java  BookService.java  HelloCon  
1 <beans xmlns="http://www.springframework.org/schema/beans"  
2     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
3     xmlns:context="http://www.springframework.org/schema/context"  
4     xsi:schemaLocation="http://www.springframework.org/schema/beans  
5         http://www.springframework.org/schema/beans/spring-bea  
6         http://www.springframework.org/schema/context  
7         https://www.springframework.org/schema/context/spring-  
8  
9     <bean id="bookService" class="vn.com.t3h.service.impl.BookServiceImpl" />
```


2.2 context:component-scan

- Để tự động phát hiện các lớp như @Controller, @Service, bạn có thể cấu hình component-scan trong dispatcher-servlet.xml như sau:

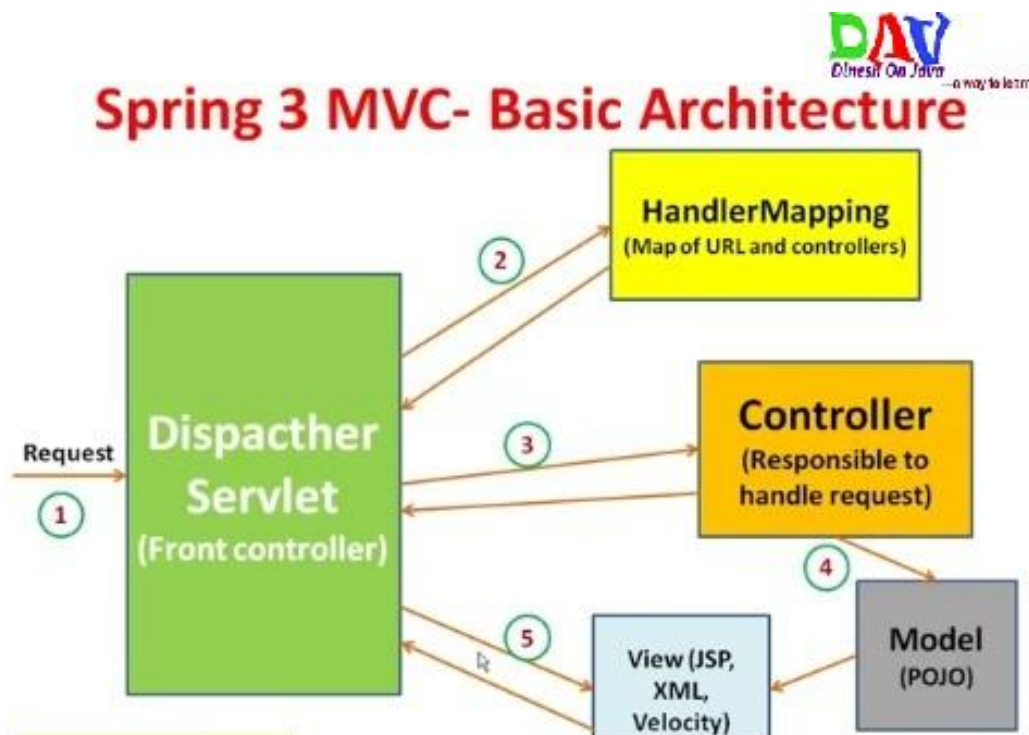


```
1 <beans xmlns="http://www.springframework.org/schema/beans"
2       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3       xmlns:context="http://www.springframework.org/schema/context"
4       xsi:schemaLocation="http://www.springframework.org/schema/beans
5                           http://www.springframework.org/schema/beans/spring-beans.xsd
6                           http://www.springframework.org/schema/context
7                           https://www.springframework.org/schema/context/spring-context.xsd"
8
9       <bean id="bookService" class="vn.com.t3h.service.impl.BookServiceImpl" />
10      <!-- Cấu hình component-scan để Spring tìm kiếm các controller -->
11      <context:component-scan base-package="vn.com.t3h.controller" />
12
```

- Điều này giúp Spring tự động quét các lớp trong package vn.com.t3h.controller và đăng ký chúng như là các bean.

III) Dispatcher-servlet trong spring

- DispatcherServlet là thành phần trung tâm trong Spring MVC. Nó nhận tất cả các yêu cầu HTTP và phân phối chúng đến các Controller thích hợp để xử lý. Sau khi Controller xử lý xong, DispatcherServlet sẽ quyết định view nào sẽ hiển thị kết quả.



III) Dispatcher-servlet trong spring

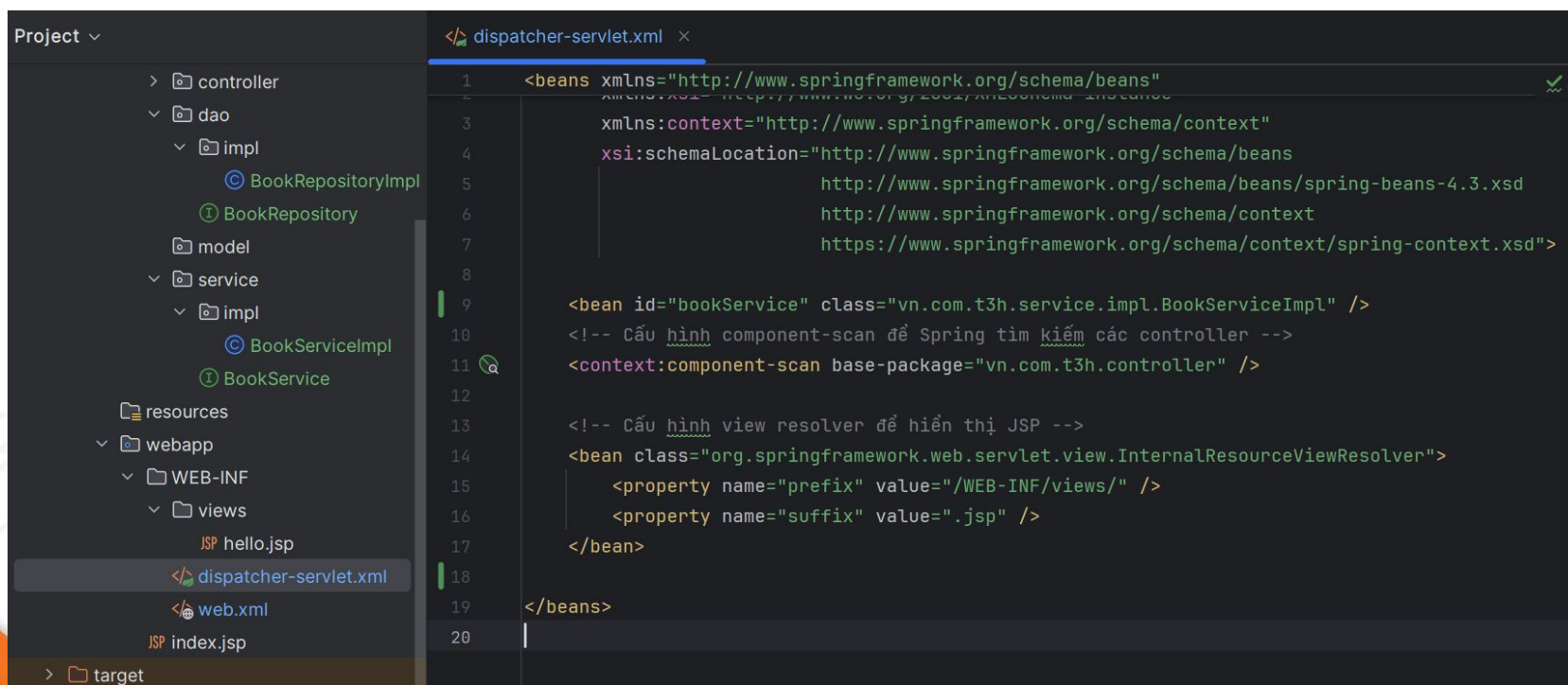
- Cấu hình trong web.xml
- Khi người dùng gửi yêu cầu HTTP, DispatcherServlet sẽ nhận yêu cầu và tìm kiếm Controller thích hợp để xử lý.

```
web.xml x JSP hello.jsp dispatcher-servlet.xml BookRepository.java BookRepositoryImp
1 <web-app xmlns="http://java.sun.com/xml/ns/javaee"
2   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3   xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
4     http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd"
5   version="3.0">
6
7   <servlet>
8     <servlet-name>dispatcher</servlet-name>
9     <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
10    <init-param>
11      <param-name>contextConfigLocation</param-name>
12      <param-value>/WEB-INF/dispatcher-servlet.xml</param-value>
13    </init-param>
14    <load-on-startup>1</load-on-startup>
15  </servlet>
16
17  <servlet-mapping>
18    <servlet-name>dispatcher</servlet-name>
19    <url-pattern>/</url-pattern>
20  </servlet-mapping>
21 </web-app>
22
```

III) Dispatcher-servlet trong spring

Cấu hình trong dispatcher-servlet.xml

- File dispatcher-servlet.xml là nơi cấu hình các thành phần quan trọng trong ứng dụng Spring MVC, bao gồm component-scan và view resolver.



```
Project ▾
  > controller
  > dao
  > impl
    > BookRepositoryImpl
    > BookRepository
  > model
  > service
  > impl
    > BookServiceImpl
    > BookService
  > resources
  > webapp
  > WEB-INF
  > views
    > JSP hello.jsp
    > dispatcher-servlet.xml
    > web.xml
    > JSP index.jsp
  > target

dispatcher-servlet.xml x
1 <beans xmlns="http://www.springframework.org/schema/beans"
2   xmlns:xsi="http://www.springframework.org/2001/XMLSchema-instance"
3   xmlns:context="http://www.springframework.org/schema/context"
4   xsi:schemaLocation="http://www.springframework.org/schema/beans
5     http://www.springframework.org/schema/beans/spring-beans-4.3.xsd
6     http://www.springframework.org/schema/context
7     https://www.springframework.org/schema/context/spring-context.xsd">
8
9   <bean id="bookService" class="vn.com.t3h.service.impl.BookServiceImpl" />
10  <!-- Cấu hình component-scan để Spring tìm kiếm các controller -->
11  <context:component-scan base-package="vn.com.t3h.controller" />
12
13  <!-- Cấu hình view resolver để hiển thị JSP -->
14  <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
15    <property name="prefix" value="/WEB-INF/views/" />
16    <property name="suffix" value=".jsp" />
17  </bean>
18
19 </beans>
20
```


III) Dispatcher-servlet trong spring

Giải thích cấu hình trong dispatcher-servlet.xml:

1.component-scan:

Cấu hình này giúp Spring quét package vn.com.t3h.controller để tìm các lớp có annotation @Controller. Spring sẽ tự động đăng ký các lớp này như là các bean và cho phép chúng xử lý yêu cầu HTTP.

Ví dụ về controller

```
<!-- Cấu hình component-scan để Spring tìm kiếm các controller -->  
<context:component-scan base-package="vn.com.t3h.controller" />
```

```
5  
6 @Controller  
7 public class HelloController {  
8  
9     @RequestMapping("/hello")  
10    public String sayHello() {  
11        return "hello"; // tên file JSP sẽ là hello.jsp  
12    }  
13 }  
14
```


III) Dispatcher-servlet trong spring

2. InternalResourceViewResolver:

- Cấu hình ViewResolver cho phép Spring xác định cách tìm kiếm các trang JSP sau khi Controller xử lý xong yêu cầu.
- **prefix:** Phần đầu của đường dẫn đến JSP.
- **suffix:** Phần đuôi của file JSP (ở đây là .jsp).
- Khi Controller trả về tên view là "hello", Spring sẽ tìm kiếm file JSP tại đường dẫn /WEB-INF/views/hello.jsp.

```
<!-- Cấu hình view resolver để hiển thị JSP -->
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <property name="prefix" value="/WEB-INF/views/" />
    <property name="suffix" value=".jsp" />
</bean>
```

IV) Controller trong Spring MVC

- Controller trong Spring MVC giúp xử lý các yêu cầu HTTP và trả về dữ liệu hoặc view. Các phương thức trong Controller có thể được ánh xạ với các yêu cầu HTTP qua annotation `@RequestMapping`.

```
5
6
7 public class HelloController {
8
9     @RequestMapping("/hello")
10    public String sayHello() {
11        return "hello"; // tên file JSP sẽ là hello.jsp
12    }
13 }
14
```

Tổng kết

- 1.component-scan tự động quét các lớp Controller và đăng ký chúng vào Spring Container.
2. InternalResourceViewResolver cấu hình cách thức Spring tìm và trả về các trang JSP sau khi Controller trả về tên view.
3. Các bean có thể được định nghĩa trong dispatcher-servlet.xml để Spring quản lý và tiêm phụ thuộc.
- 4.Controller là nơi xử lý các yêu cầu HTTP và trả về dữ liệu cho View.

